Universidad del País Vasco Euskal Herriko Unibertsitatea
INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

**Informatika Ingeniaritzako Fakultatea**

**S**oftware **I**ngeniaritza **2**

# *2.Entrega*

# *Bad Smell Errefaktorizazioak*

---

**Egileak:** Iker Garcia, Ibai Fuertes eta Aitzol Velez de Mendizabal

**Data:** 18/10/26

# 1. Errefaktorizazioa (Iker Garcia)

## 1.1.-"Write short units of code"

### A) Hasierako Kodea

```
public void acceptReclamation(boolean b, long errekId) {
    try {
        db.getTransaction().begin();
        Erreklamazioa errek = db.find(Erreklamazioa.class, errekId);
        if (errek != null) {
            if (b) {
                errek.setOnartua(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.Onartua"));
                // erosleari dirua itzuli@WebMethod public List<Sale> obtainList(String
                // usermail)
                Double diru = (double) errek.getSale().getPrice();

                errek.getErreklamatzenDuena().setDiruTotala(errek.getErreklamatzenDuena().getDiruTotala() + diru);
                errek.getErreklamatzenDuena().removeBought(errek.getSale());
                String desk = errek.getSale().getTitle();
                errek.getErreklamatzenDuena().addMugimendua(UtilDate.trim(new Date()), desk, diru);

                // seller-ari dirua kendu

            } else {
                errek.setOnartua(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.EzOnartua"));
                errek.getSale().setErreklamatuta(false);
                // dirua ez itzuli
            }

            db.getTransaction().commit();
        } else {
            db.getTransaction().rollback();
        }
    } catch (Exception e) {
        if (db.getTransaction() != null && db.getTransaction().isActive()) {
            db.getTransaction().rollback(); // Aldaketak desegiten ditu errek null bada

        }
        throw e;
    }
}
```

B) Errefaktorizatutako Kodea

```
public void acceptReclamation(boolean b, long errekId) {
    try {
        db.getTransaction().begin();
        Erreklamazioa errek = db.find(Erreklamazioa.class, errekId);
        if (errek != null) {
            onartutaJarri(b, errek);

            db.getTransaction().commit();
        } else {
            db.getTransaction().rollback();
        }
    } catch (Exception e) {
        if (db.getTransaction() != null && db.getTransaction().isActive()) {
            db.getTransaction().rollback(); // Aldaketak desegiten ditu errek null bada

        }
        throw e;
    }
}

private void onartutaJarri(boolean b, Erreklamazioa errek) {
    if (b) {
        errek.setOnartua(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.Onartua"));
        // erosleari dirua itzuli@WebMethod public List<Sale> obtainList(String
        // usermail)
        Double diru = (double) errek.getSale().getPrice();

        errek.getErreklamatzenDuena().setDiruTotala(errek.getErreklamatzenDuena().getDiruTotala() + diru);
        errek.getErreklamatzenDuena().removeBought(errek.getSale());
        String desk = errek.getSale().getTitle();
        errek.getErreklamatzenDuena().addMugimendua(UtilDate.trim(new Date()), desk, diru);

        // seller-ari dirua kendu

    } else {
        errek.setOnartua(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.EzOnartua"));
        errek.getSale().setErreklamatuta(false);
        // dirua ez itzuli
    }
}
```

C) Egindako Errefaktorizazioaren Deskribapena

Extract metod erabiliz acceptReclamation metodoan zegoen if bat beste metodo berri batera bilakatu dut. Metodo horretan ezarriko zaio erreklamazioari onartuta dagoen edo ez. Horrela, lortzen da kodigoa 15 lerro baino gutxiago izatea

## 1.2.-"Write simple units of code"

## A) Hasierako Kodea

```
public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail,
        File file) throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException,
        ParamNullException {

    System.out.println(">> DataAccess: createProduct=> title= " + title + " seller=" + sellerEmail);
    try {

        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new MustBeLaterThanTodayException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }
        if (file == null)
            throw new FileNotUploadedException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorFileNotUploadedException"));

        db.getTransaction().begin();

        Seller seller = db.find(Seller.class, sellerEmail);
        if (seller.doesSaleExist(title)) {
            db.getTransaction().commit();
            throw new SaleAlreadyExistException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
        }

        Sale sale = seller.addSale(title, description, status, price, pubDate, file);
        // next instruction can be obviated

        db.persist(seller);
        db.getTransaction().commit();
        System.out.println("sale stored " + sale + " " + seller);

        System.out.println("hasta aqui");

        return sale;
    } catch (NullPointerException e) {
        e.printStackTrace();
        // TODO Auto-generated catch block
        db.getTransaction().commit();
        return null;
    }

}
```

## B) Errefaktorizatutako Kodea

```
public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail,
        File file) throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException,
        ParamNullException {

    System.out.println(">> DataAccess: createProduct=> title= " + title + " seller=" + sellerEmail);
    try {

        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new MustBeLaterThanTodayException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }
        isFileNull(file);

        db.getTransaction().begin();

        Seller seller = db.find(Seller.class, sellerEmail);
        if (seller.doesSaleExist(title)) {
            db.getTransaction().commit();
            throw new SaleAlreadyExistException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
        }

        Sale sale = seller.addSale(title, description, status, price, pubDate, file);
        // next instruction can be obviated

        db.persist(seller);
        db.getTransaction().commit();
        System.out.println("sale stored " + sale + " " + seller);

        System.out.println("hasta aqui");

        return sale;
    } catch (NullPointerException e) {
        e.printStackTrace();
        // TODO Auto-generated catch block
        db.getTransaction().commit();
        return null;
    }

}

  private void isFileNull(File file) throws FileNotUploadedException {
      if (file == null)
          throw new FileNotUploadedException(
                  ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorFileNotUploadedException"));
  }
```

## C) Egindako Errefaktorizazioaren Deskribapena

Ikusten denez createSale klasearen hasierako kodean bere konplexutasun ziklomatikoa 5 da eta gehienez 4 izatea lortu behar dugu. Horretarako berriro ere ExtractMethod erabiliz if bat metodo batean bilakatu dut eta horrela metodo horrek ez du 5-eko konplexutasun ziklomatikoa izango baizik eta 4koa.

# 1.3.-"Duplicate Code"

A) Hasierako Kodea

```
public void diruaSartu(String mail, double dirua) {
    Seller u = db.find(Seller.class, mail);
    db.getTransaction().begin();
    Date today = UtilDate.trim(new Date());
    u.diruaSartu(dirua, today);
    db.getTransaction().commit();

}

public void diruaAtera(String mail, double dirua) {
    Seller u = db.find(Seller.class, mail);
    db.getTransaction().begin();
    Date today = UtilDate.trim(new Date());
    u.diruaAtera(dirua, today);
    db.getTransaction().commit();
```

B) Errefaktorizatutako Kodea

```
private void diruaMugimendua(String mail,double dirua,boolean sartu) {

    Seller u = db.find(Seller.class, mail);
    db.getTransaction().begin();
    Date today = UtilDate.trim(new Date());

    if (sartu) {
        u.diruaSartu(dirua, today);
    } else {
        u.diruaAtera(dirua, today);
    }
    db.getTransaction().commit();
}

public void diruaSartu(String mail, double dirua) {
    diruaMugimendua(mail, dirua, true);
}

public void diruaAtera(String mail, double dirua) {
    diruaMugimendua(mail, dirua, false);
}
```

C) Egindako Errefaktorizazioaren Deskribapena

Duplikatutako kodea murrizteko, diruaAtera eta diruaSartu funtzioak zituzten errepikatutako lerroak diruaMugimendua funtzioan sartu ditut eta horrela duplikazioa murriztuko da.

## 1.4.-"Keep Unit Interfaces Small"

A) Hasierako Kodea

```
public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail,
        File file) throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException,
        ParamNullException {

    System.out.println(">> DataAccess: createProduct=> title= " + title + " seller=" + sellerEmail);
    try {

        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new MustBeLaterThanTodayException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }
        isFileNull(file);

        db.getTransaction().begin();

        Seller seller = db.find(Seller.class, sellerEmail);
        if (seller.doesSaleExist(title)) {
            db.getTransaction().commit();
            throw new SaleAlreadyExistException(
                    Duplication
                    ResourceBundle.getBundle( I "Etiquetas").getString("DataAccess.SaleAlreadyExist"));
        }

        Sale sale = seller.addSale(title, description, status, price, pubDate, file);
        // next instruction can be obviated

        db.persist(seller);
        db.getTransaction().commit();
        System.out.println("sale stored " + sale + " " + seller);

        System.out.println("hasta aqui");

        return sale;
    } catch (NullPointerException e) {
        e.printStackTrace();
        // TODO Auto-generated catch block
        db.getTransaction().commit();
        return null;
    }

}
```

## B) Errefaktorizatutako Kodea

```
package dataAccess;
import java.io.File;
import java.util.Date;

public class CreateSaleData {
    private String title;
    private String description;
    private int status;
    private float price;
    private Date pubDate;
    private String sellerEmail;
    private File file;


    public CreateSaleData(String title, String description, int status, float price, Date pubDate, String sellerEmail, File file) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
        this.pubDate = pubDate;
        this.sellerEmail = sellerEmail;
        this.file = file;
    }

    public String getTitle() { return title; }
    public String getDescription() { return description; }
    public int getStatus() { return status; }
    public float getPrice() { return price; }
    public Date getPubDate() { return pubDate; }
    public String getSellerEmail() { return sellerEmail; }
    public File getFile() { return file; }
}
```

```
public Sale createSale(CreateSaleData param) throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException,
        ParamNullException {

        System.out.println(">> DataAccess: createProduct=> title= " + param.getTitle() + " seller=" + param.getSellerEmail());
        try {
            if (param.getPubDate().before(UtilDate.trim(new Date()))) {
                throw new MustBeLaterThanTodayException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
            }
            isFileNull(param.getFile());

            db.getTransaction().begin();

            Seller seller = db.find(Seller.class, param.getSellerEmail());
            if (seller.doesSaleExist(param.getTitle())) {
                db.getTransaction().commit();
                throw new SaleAlreadyExistException(
                    ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
            }


            Sale sale = seller.addSale(param.getTitle(), param.getDescription(), param.getStatus(),
                                       param.getPrice(), param.getPubDate(), param.getFile());

            db.persist(seller);
            db.getTransaction().commit();
            System.out.println("sale stored " + sale + " " + seller);

            return sale;
        } catch (NullPointerException e) {
            e.printStackTrace();
            db.getTransaction().commit();
            return null;
        }


}

@WebMethod
 public Sale createSale(String title, String description, int status, float price, Date pubDate, String sellerEmail, File file)
         throws  FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException, ParamNullException {
     dbManager.open();
     CreateSaleData param = new CreateSaleData(title, description, status, price, pubDate, sellerEmail, file);
     Sale product=dbManager.createSale(param);
     dbManager.close();
     return product;
}

/**
```

C) Egindako Errefaktorizazioaren Deskribapena

Ikusten denez 4 parametro edo gutxiago funtzioan sartzeko, klase berri bat sortu dut CreateSaleData izenekoa. Honen bidez orain bakarrik parametro bat pasatzen da createSale metodoan. Gainera, BLFacadeImplementation clasea aldatu dut bertan zegoen createSale metodoa param aldagaia jasotzeko.

Hau egiteko Building Maintainable Software liburuan agertzen diren pausuak jarraitu ditut.

# 2. Errefaktorizazioa (Ibai Fuertes)

## 2.1.-"Write short units of code"

### A) Hasierako Kodea

```
public boolean karrituraEraman(String mail, int salenum) {
    Seller user = db.find(Seller.class, mail);
    Sale sale = db.find(Sale.class, salenum);
    ErosketaAnitza ea = user.getErosketaAnitza();
    boolean b = true;
    db.getTransaction().begin();
    try {
        if (ea == null) {
            ea = user.createErosketaAnitza(sale);
        } else {// konprobatu pertsona berdinaren produktuak direla
            if (sale.getSeller() != ea.getSeller()) {
                throw new UserAlreadyExistException();// Ez dut beste bat egingo bakarrik honetarako
            }
        }
        db.persist(ea);
        ea.addSales(sale);
        sale.setAnitza(ea);
        ea.gehituPrezioa(sale.getPrice());
    } catch (UserAlreadyExistException e) {
        b = false;
    }
    db.getTransaction().commit();
    return b;
}
```

### B) Errefaktorizatutako Kodea

```
public boolean karrituraEraman(String mail, int salenum) {
    Seller user = db.find(Seller.class, mail);
    Sale sale = db.find(Sale.class, salenum);
    ErosketaAnitza ea = user.getErosketaAnitza();
    boolean b = true;
    db.getTransaction().begin();
    try {
        if (ea == null) {
            ea = user.createErosketaAnitza(sale);
        } else {// konprobatu pertsona berdinaren produktuak direla
            if (sale.getSeller() != ea.getSeller()) {
                throw new UserAlreadyExistException();// Ez dut beste bat egingo bakarrik honetarako
            }
        }
        db.persist(ea);
        addSaleToErosketaAnitza(ea, sale);
    } catch (UserAlreadyExistException e) {
        b = false;
    }
    db.getTransaction().commit();
    return b;
}
public void addSaleToErosketaAnitza(ErosketaAnitza ea, Sale sale)
{
    ea.addSales(sale);
    sale.setAnitza(ea);
    ea.gehituPrezioa(sale.getPrice());
}
```

### C) Egindako Errefaktorizazioaren Deskribapena

KarrituraEraman metodoak kode lerro gehigi zituen, horregatik beste metodo txiki bat sortu egin dugu metodoa errexago ulertzeko. Metodo berri honetan, erosketa anitz bat eta sale bat emanda egin beharreko aldaketak egiten dira, sale hori erosketa anitzean sartuz.

## 2.2.-"Write simple units of code"

**Oharra:** berez metodo honek ez du behar errefaktorizaziorik, baino ez dugunez beste metodo errefaktorizazio mota hau behar duena, metodo hau aukeratu dugu limitean dagoenez.

### A) Hasierako Kodea

```
public boolean karrituraEraman(String mail, int salenum) {
    Seller user = db.find(Seller.class, mail);
    Sale sale = db.find(Sale.class, salenum);
    ErosketaAnitza ea = user.getErosketaAnitza();
    boolean b = true;
    db.getTransaction().begin();
    try {
        if (ea == null) {
            ea = user.createErosketaAnitza(sale);
        } else {// konprobatu pertsona berdinaren produktuak direla
            if (sale.getSeller() != ea.getSeller()) {
                throw new UserAlreadyExistException();// Ez dut beste bat egingo bakarrik honetarako
            }
        }
        db.persist(ea);
        addSaleToErosketaAnitza(ea, sale);
    } catch (UserAlreadyExistException e) {
        b = false;
    }
    db.getTransaction().commit();
    return b;
}
```

### B) Errefaktorizatutako Kodea

```
public boolean karrituraEraman(String mail, int salenum) {
    Seller user = db.find(Seller.class, mail);
    Sale sale = db.find(Sale.class, salenum);
    ErosketaAnitza ea = user.getErosketaAnitza();
    boolean b = true;
    db.getTransaction().begin();
    try {
        if (ea == null) {
            ea = user.createErosketaAnitza(sale);
        } else {// konprobatu pertsona berdinaren produktuak direla
            konprobatuSeller(ea, sale);
        }
        db.persist(ea);
        addSaleToErosketaAnitza(ea, sale);
    } catch (UserAlreadyExistException e) {
        b = false;
    }
    db.getTransaction().commit();
    return b;
}
public void konprobatuSeller(ErosketaAnitza ea, Sale sale) throws UserAlreadyExistException
{
    if (sale.getSeller() != ea.getSeller()) {
        throw new UserAlreadyExistException();// Ez dut beste bat egingo bakarrik honetarako
    }
}
```

### C) Egindako Errefaktorizazioaren Deskribapena

Metodoaren konplexutasun ziklomatikoa jaisteko askmoz, metodoak beste metodo berri bat deitzen du begiratzeko sale batek eta erosketa anitz batek saltzaile bera duten, desberdinak badira orduan UserAlreadyExistException igoko du.

## 2.3.-"Duplicate Code"

### A) Hasierako Kodea

```
public List<Erreklamazioa> getJrerreklamazioak(String mail) {
    Seller user = db.find(Seller.class, mail);
    List<Erreklamazioa> jErreList = user.getJarritakoErreklamazioak();
    return jErreList;

}

public List<Erreklamazioa> getJserreklamazioak(String mail) {
    Seller user = db.find(Seller.class, mail);
    List<Erreklamazioa> jErreList = user.getJasotakoErreklamazioak();
    return jErreList;

}
```

B) Errefaktorizatutako Kodea

```
public List<Erreklamazioa> getErreklamazioak(String mail, boolean jasotako) {
    Seller user = db.find(Seller.class, mail);
    List<Erreklamazioa> jErreList;
    if (!jasotako)
    {
        jErreList = user.getJarritakoErreklamazioak();
    }
    else
    {
        jErreList = user.getJasotakoErreklamazioak();
    }
    return jErreList;
}
```

C) Egindako Errefaktorizazioaren Deskribapena

Bi metodo horiek nahiko berdinak direnez metodo bakar batean elkartu ditugu boolear baten mende egongo dena, bata ala bestearen kodea exekutatzeko.

## 2.4.- "Keep Unit Interfaces Small"

**Oharra:** berez metodo honek ez du behar errefaktorizaziorik, baino ez dugunez beste metodo errefaktorizazio mota hau behar duena, metodo hau aukeratu dugu limitean dagoenez.

A) Hasierako Kodea

```
public void erreklamazioaJarri(String zuremail, int saleNumber, String azalpena) {
    Seller user = db.find(Seller.class, zuremail);
    Sale sale = db.find(Sale.class, saleNumber);
    db.getTransaction().begin();
    sale.setErreklamatuta(true);
    Erreklamazioa errek = new Erreklamazioa(sale, UtilDate.trim(new Date()), azalpena, user);
    user.addJarritakoErreklamazioak(errek);
    sale.getSeller().addJasotakoErreklamazioak(errek);
    db.getTransaction().commit();
}
```

B) Errefaktorizatutako Kodea

```java
public class ErreklamazioaJarriData {
    private String mail;
    private int saleNumber;
    private String azalpena;

    public ErreklamazioaJarriData(String m, int sNum, String azal)
    {
        mail = m;
        saleNumber = sNum;
        azalpena = azal;
    }

    public String getMail() { return mail; }
    public int getSaleNumber() { return saleNumber; }
    public String getAzalpena() { return azalpena; }
}
```

```java
public void erreklamazioaJarri(ErreklamazioaJarriData data) {
    Seller user = db.find(Seller.class, data.getMail());
    Sale sale = db.find(Sale.class, data.getSaleNumber());
    db.getTransaction().begin();
    sale.setErreklamatuta(true);
    Erreklamazioa errek = new Erreklamazioa(sale, UtilDate.trim(new Date()), data.getAzalpena(), user);
    user.addJarritakoErreklamazioak(errek);
    sale.getSeller().addJasotakoErreklamazioak(errek);
    db.getTransaction().commit();
}
```

## C) Egindako Errefaktorizazioaren Deskribapena

Metodoak parametro gehiegi dituenez, klase berri bat sortu dugu ErreklamazioJarriData izenekoa parametro bezala sartuko dena metodo horretan. Klase honetan egongo da metodoak behar duen informazio guztia.

# 3. Errefaktorizazioa (Aitzol Velez de Mendizabal)

## 3.1.-"Write short units of code"

### A) Hasierako Kodea

```
public boolean chatIreki(String mail, String mail2) {
    // true ondo joan bada
    boolean b = true;

    Seller nor = db.find(Seller.class, mail);
    Seller nori = db.find(Seller.class, mail2);
    TypedQuery<Chat> query = db.createQuery(
            "SELECT s FROM Chat s WHERE (user1=?1 AND user2=?2) OR (user1=?2 AND user2=?1)", Chat.class);
    query.setParameter(1, nor);
    query.setParameter(2, nori);
    List<Chat> list = query.getResultList();// begiratu hasieratu ez dela dagoeneko
    db.getTransaction().begin();
    if (!list.isEmpty()) {
        b = false;
    } else {// listan ez dago, hau da, dagoeneko ez da ireki
        if (nor == null || nori == null) {
            // mail-a gaizki dago
            b = false;
        } else {
            Chat c = nor.createChat(nori);
            db.persist(c);
        }
    }
    db.getTransaction().commit();
    return b;
}
```

### B) Errefaktorizatutako Kodea

```
public boolean chatIreki(String mail, String mail2) {
    // true ondo joan bada
    Seller nor = db.find(Seller.class, mail);
    Seller nori = db.find(Seller.class, mail2);
    boolean b = !chatExists(nor, nori);

    db.getTransaction().begin();
    if (b) {
        b = createChat(nor, nori);
    } else {// listan ez dago, hau da, dagoeneko ez da ireki
        return createChat(nor, nori);
    }
    db.getTransaction().commit();
    return b;
}

private boolean chatExists(Seller nor, Seller nori) {
    TypedQuery<Chat> query = db.createQuery(
            "SELECT s FROM Chat s WHERE (user1=?1 AND user2=?2) OR (user1=?2 AND user2=?1)", Chat.class);
    query.setParameter(1, nor);
    query.setParameter(2, nori);
    return !query.getResultList().isEmpty();
}

private boolean createChat(Seller nor, Seller nori) {
    if (nor == null || nori == null) {
        // mail-a gaizki dago
        return false;
    } else {
        db.persist(nor.createChat(nori));
        return true;
    }
}
```

### C) Egindako Errefaktorizazioaren Deskribapena

chatIreki() metodoak erabiltzaileak eta txat-a existitzen diren eta mailak zuzenak diren begiratzen du eta txat-a sortzen du. Horrela 23 kode lerro okupatzen ditu eta, laburtzeko, txat-a existitzen den beste metodo batean begiratzen du eta txat-a beste batean sortzen du.

## 3.2.-"Write simple units of code"

### A) Hasierako Kodea

```
private String check_fields_Errors() {

    try {
        if ((fieldTitle.getText().length() == 0) || (fieldDescription.getText().length() == 0)
                || (jTextFieldPrice.getText().length() == 0))
            return ResourceBundle.getBundle(ETIQUETAS).getString("CreateSaleGUI.ErrorQuery");
        else {

            // trigger an exception if the introduced string is not a number
            float price = Float.parseFloat(jTextFieldPrice.getText());
            if (price <= 0)
                return ResourceBundle.getBundle(ETIQUETAS).getString("CreateSaleGUI.PriceMustBeGreaterThan0");

            else
                return null;
        }
    } catch (java.lang.NumberFormatException e1) {

        return ResourceBundle.getBundle(ETIQUETAS).getString("CreateSaleGUI.ErrorNumber");
    } catch (Exception e1) {
        return null;

    }
}
```

### B) Errefaktorizatutako Kodea

```
private String check_fields_Errors() {

    try {
        if (isAnyFieldEmpty())
            return ResourceBundle.getBundle(ETIQUETAS).getString("CreateSaleGUI.ErrorQuery");
        else {

            // trigger an exception if the introduced string is not a number
            float price = Float.parseFloat(jTextFieldPrice.getText());
            return checkPrize(price);
        }
    } catch (java.lang.NumberFormatException e1) {

        return ResourceBundle.getBundle(ETIQUETAS).getString("CreateSaleGUI.ErrorNumber");
    } catch (Exception e1) {
        return null;

    }
}

private String checkPrize(float price) {
    if (price <= 0)
        return ResourceBundle.getBundle(ETIQUETAS).getString("CreateSaleGUI.PriceMustBeGreaterThan0");

    else
        return null;
}

private boolean isAnyFieldEmpty() {
    return (fieldTitle.getText().length() == 0) || (fieldDescription.getText().length() == 0)
            || (jTextFieldPrice.getText().length() == 0);
}
```

### C) Egindako Errefaktorizazioaren Deskribapena

CreateSaleGUI klaseko check_fields_Errors() metodoak 6 adar-puntu ditu (2 if, 2 || eta 2 catch), horregatik 2 OR-ak beste metodo batera atera dira eta berdina egin da if(price <= 0) baldintzarekin.

## 3.3.-"Duplicate Code"

### A) Hasierako Kodea

```
public Seller isUserLogin(String username, String password) {
    TypedQuery<Seller> query = db.createQuery("SELECT s FROM Seller s WHERE s.name=?1 AND s.password=?2",
            Seller.class);
    query.setParameter(1, username);
    query.setParameter(2, password);
    if (!query.getResultList().isEmpty()) {
        return query.getResultList().get(0);
    } else {
        return null;
    }
}

public Admin isAdminLogin(String nan, String password) {
    TypedQuery<Admin> query = db.createQuery("SELECT s FROM Admin s WHERE s.NAN=?1 AND s.pasahitza=?2",
            Admin.class);
    query.setParameter(1, nan);
    query.setParameter(2, password);
    if (!query.getResultList().isEmpty()) {
        return query.getResultList().get(0);
    } else {
        return null;
    }
}
```

### B) Errefaktorizatutako Kodea

```
private <T> T findByCredentials(Class<T> type, String jpql, String user, String password) {
    TypedQuery<T> query = db.createQuery(jpql, type);
    query.setParameter(1, user);
    query.setParameter(2, password);
    if (!query.getResultList().isEmpty()) {
        return query.getResultList().get(0);
    } else {
        return null;
    }
}

public Seller isUserLogin(String username, String password) {
    return findByCredentials(Seller.class, "SELECT s FROM Seller s WHERE s.name=?1 AND s.password=?2", username,
            password);
}

public Admin isAdminLogin(String nan, String password) {
    return findByCredentials(Admin.class, "SELECT s FROM Seller s WHERE s.NAN=?1 AND s.pasahitza=?2", nan,
            password);
}
```

### C) Egindako Errefaktorizazioaren Deskribapena

isUserLogin() eta isAdminLogin() metodoak oso antzekoak ziren eta beraz findByCredentials() metodoarekin bateratu dira, metodo bakoitzak jpql eskaera propioa bidali arren, egitura berdina jarraitzeko.

## 2.4.- "Keep Unit Interfaces Small"

### A) Hasierako Kodea

// domain.Seller

```
public Sale addSale(String title, String description, int status, float price, Date pubDate, File file) {

    Sale sale = new Sale(title, description, status, price, pubDate, file, this);
    sales.add(sale);
    return sale;
}
```

// DataAccess.createSale()

```
Sale sale = seller.addSale(param.getTitle(), param.getDescription(), param.getStatus(), param.getPrice(),
        param.getPubDate(), param.getFile());
```

B) Errefaktorizatutako Kodea

// domain.Seller

```
public Sale addSale(CreateSaleData data) {
    Sale sale = new Sale(data.getTitle(), data.getDescription(), data.getStatus(), data.getPrice(),
            data.getPubDate(), data.getFile(), this);
    sales.add(sale);
    return sale;
}
```

// DataAccess.createSale()

```
Sale sale = seller.addSale(param);
```

C) Egindako Errefaktorizazioaren Deskribapena

Seller.addSale() metodoak 6 parametro ditu eta modu honetara guztiak params parametro bakar batean elkartzen dira.